



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N.º 07

NOMBRE COMPLETO: Rosas Cañada Abraham

N.º de Cuenta: 318307866

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 19/04/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1: Agregar movimiento con teclado al helicóptero hacia adelante y atrás.

Para mover el helicóptero con teclado fue necesario modificar la clase Window del proyecto, ya que es ahí donde se registran los estados de las teclas. Se agregó una nueva variable llamada muevehelicoptero y su respectivo getter, así como el manejo de las teclas H y J dentro de handleKeys. La tecla H desplaza el helicóptero hacia adelante sobre el eje X y la tecla J lo regresa. Se utilizó el eje X en lugar del eje Z porque visualmente el helicóptero avanza de frente hacia atrás desde la perspectiva de la cámara.

Cambios en Window.h (declaración de la variable y del getter):

```
private:
    GLfloat muevex;
    GLfloat muevehelicoptero;

public:
    GLfloat getmuevex() { return muevex; }
    GLfloat getmuevehelicoptero() { return muevehelicoptero; }
```

Cambios en Window.cpp (inicialización en el constructor y manejo de teclas):

```
// En el constructor
muevex = 0.0f;
muevehelicoptero = 0.0f;

// Dentro de handleKeys junto a las teclas del coche
if (theWindow->keys[GLFW_KEY_H])
{
    theWindow->muevehelicoptero += 0.05f;
}
if (theWindow->keys[GLFW_KEY_J])
{
    theWindow->muevehelicoptero -= 0.05f;
}
```

En main.cpp se recupera el valor del desplazamiento y se aplica a la traslación del helicóptero dentro del while principal:

```
// helicóptero con movimiento por teclas H y J en eje X
GLfloat heliX = 0.0f + mainWindow.getmuevehelicoptero();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(heliX, 5.0f, 6.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Blackhawk_M.RenderModel();
```

Ejercicio 2: Luz spotlight amarilla del helicóptero que apunta al piso y se mueve con él.

Se agregó una cuarta luz del tipo SpotLight al arreglo spotLights con índice 3. El color se fijó en RGB (1.0, 1.0, 0.0) para obtener amarillo puro, la dirección inicial se apunta al piso con el vector (0, -1, 0) y el ángulo de apertura del cono se estableció en 20 grados. Para que la luz siga al helicóptero se reutilizó el método SetFlash dentro del while, igual que se hace con la linterna de la cámara y con el faro azul del coche, usando la misma variable heliX del modelo para mantener sincronizados modelo y luz.

Declaración del spotlight antes del ciclo while:

```
// spotlight amarillo del helicoptero
spotLights[3] = SpotLight(1.0f, 1.0f, 0.0f,
    0.0f, 1.0f,
    0.0f, 5.0f, 6.0f,
    0.0f, -1.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    20.0f);
spotLightCount++;
```

Actualización de la luz cada frame dentro del while para que siga al helicóptero:

```
// spotlight amarillo ligado al helicoptero apuntando al piso
glm::vec3 posSpotHeli = glm::vec3(heliX, 5.0f, 6.0f);
glm::vec3 dirSpotHeli = glm::vec3(0.0f, -1.0f, 0.0f);
spotLights[3].SetFlash(posSpotHeli, dirSpotHeli);
```



Figura 1. Helicóptero importado desde Blender con movimiento controlado por teclado (teclas H y J) sobre el eje X y spotlight amarillo ligado a su posición apuntando al piso.

Ejercicio 3: Añadir un modelo de lámpara texturizada con luz puntual blanca.

Se modeló una lámpara de mesa en Blender con sus respectivos materiales e imágenes de textura. Al exportar el modelo con File, Export, Wavefront (.obj) activando las opciones Write Materials y Export UVs, Blender generó tres archivos: Lamparatest.obj con la geometría, Lamparatest.mtl con la definición de materiales que incluye la directiva map_Kd apuntando al archivo de imagen, y la textura TGA utilizada. Los tres archivos se colocaron en las carpetas Models y Textures del proyecto respectivamente. En el código de OpenGL no fue necesario declarar una variable Texture extra para la lámpara, ya que la clase Model lee automáticamente el archivo .mtl asociado al .obj y carga la imagen referenciada por map_Kd mediante stb_image, replicando el mismo flujo que se utilizó con el coche.

Declaración y carga del modelo:

```
Model Lamparatest_M;  
  
// dentro de main  
Lamparatest_M = Model();  
Lamparatest_M.LoadModel("Models/Lamparatest.obj");
```

Para que la luz puntual blanca quedara ligada a la lámpara y ambas se movieran juntas si se decide reubicarlas, se definió una única variable posLampara que

alimenta tanto al translate del modelo como a la posición de la luz. Además, la luz se reconstruye dentro del while cada frame tomando esa variable, lo que replica el comportamiento de SetFlash que usan los spotlights pero para una luz puntual:

```
// posicion de la lampara
glm::vec3 posLampara = glm::vec3(-6.0f, -0.5f, -8.0f);

// luz puntual blanca ligada a la lampara
pointlights[1] = PointLight(1.0f, 1.0f, 1.0f,
    0.2f, 3.0f,
    posLampara.x, posLampara.y + 1.0f, posLampara.z,
    1.0f, 0.7f, 1.8f);

// render de la lampara usando la misma variable
model = glm::mat4(1.0);
model = glm::translate(model, posLampara);
model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
Lamparatem_M.RenderModel();
```

El offset de +1.0 en la coordenada Y de la luz coloca el foco a la altura del bombillo de la lámpara y no en la base. Los coeficientes 1.0, 0.7, 1.8 corresponden a los términos constante, lineal y exponencial de la atenuación, los cuales controlan qué tan rápido se apaga la luz con la distancia y por lo tanto el diámetro visual del círculo iluminado.



Figura 2. Lámpara texturizada importada desde Blender con luz puntual blanca emitiendo desde la parte superior del modelo..

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Aprender a manejar los tres tipos de luz: la práctica implicó usar al mismo tiempo luz direccional, luz puntual y spotlight, cada una con parámetros distintos. La luz direccional sólo necesita un vector de dirección porque simula el sol y no tiene posición. La luz puntual emite en todas direcciones desde un punto del espacio y se atenúa con la distancia. El spotlight es como una linterna, tiene posición, dirección y ángulo de apertura. Entender cuándo usar cada una fue clave para decidir que la lámpara debía iluminar todo alrededor con una puntual y el helicóptero debía enfocar al piso con un spotlight.

Los coeficientes de atenuación constante, lineal y exponencial: inicialmente la luz puntual de la lámpara se veía con un radio demasiado grande, más grande que el propio modelo, dando un efecto poco natural. El coeficiente constante se deja fijo en 1.0, el lineal hace que la luz decaiga de forma gradual con la distancia y el exponencial corta la luz más bruscamente en los bordes. Tras probar los valores tradicionales 1.0, 0.045, 0.0075 el círculo seguía siendo muy amplio, por lo que se cambiaron a 1.0, 0.7, 1.8 que concentran la luz en un radio proporcional al tamaño de la lámpara.

Ligar una luz puntual a un objeto: la clase SpotLight dispone del método SetFlash que actualiza posición y dirección en tiempo de ejecución, pero PointLight no ofrece un equivalente directo en la versión del curso. La solución fue reconstruir la luz con el constructor PointLight dentro del while cada frame, usando una variable compartida con el translate del modelo, lo que en la práctica produce el mismo efecto que un SetFlash pero para luces omnidireccionales.

Iluminación irregular en la pantalla de la lámpara: al renderizar la lámpara con la luz activada, algunas caras de la pantalla aparecían correctamente iluminadas mientras que otras se veían oscuras formando un patrón de tiras alternadas. El problema no provenía del código de iluminación sino de las normales del modelo, que estaban invertidas en algunas caras por cómo se hizo el modelado y duplicado en Blender. La solución consistió en seleccionar el objeto en modo Edit dentro de Blender, seleccionar toda la malla con la tecla A y aplicar Recalculate Outside con el atajo Shift+N, lo que reorientó todas las normales hacia afuera. Después se exportó el OBJ nuevamente y se reemplazó el archivo en la carpeta Models del proyecto.

3.- Conclusión:

Esta práctica permitió integrar en un solo escenario los tres tipos de luz que se estudian en computación gráfica y comprender cómo cada una responde a parámetros distintos según el fenómeno físico que intenta simular. Trabajar con el helicóptero reforzó el concepto de ligar una luz al movimiento de un modelo mediante una variable compartida, evitando duplicar lógica y manteniendo sincronizados objeto e iluminación en tiempo de ejecución. Por su parte, la incorporación de la lámpara demostró que el flujo de trabajo entre Blender y OpenGL depende fuertemente del archivo .mtl como puente entre la geometría exportada y las imágenes de textura, y que la clase Model del curso se encarga internamente de resolver toda esa cadena sin necesidad de declarar texturas adicionales en el código principal. Los problemas encontrados con los coeficientes de atenuación y con las normales del modelo sirvieron para comprender que la calidad visual del render no sólo depende del shader, sino también de preparar adecuadamente los modelos en el software de modelado antes de exportarlos. En conjunto, los tres ejercicios permitieron pasar de una escena con objetos iluminados de manera uniforme a una escena con iluminación dirigida, dinámica y con comportamiento físicamente coherente.

Bibliografía:

- Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3 (8th ed.). Addison-Wesley Professional.
- Blender Foundation. (2025). Blender 4.x Reference Manual: Modeling, Materials and Export. <https://docs.blender.org>
- The Khronos Group. (2024). OpenGL 4.6 Core Profile Specification. <https://registry.khronos.org/OpenGL/specs/gl/glspec46.core.pdf>